

Edwin Rojas

1. Nombra 3 métodos de Array y explica que hacen

forEach: recibe un callback que se va aplicar a cada item en el array

findIndex: busca y regresa el índice de un item en el array

find: busca si existe un item en el array y regresa true si lo encuentra o false si no.
(con truthy ints, -1, 1, 0);

2. Escribe el código para crear una clase user con los campos name y age, su constructor y los métodos set y get para uno de los campos.

```
class user {  
  constructor(name, age) {  
    this.name = name;  
    this.age = age;  
  }  
  get() {  
    return this.name;  
  }  
  set() {  
    this.name = name;  
  }  
}
```

3. Declara una función que espere un segundo y después imprima algo

```
function printWait() {  
  setTimeout(() => {  
    console.log("Waited 1 second!");  
  }, 1000);  
}
```

4. ¿Qué es un callback?

Es una función que se usa como argumento de otra función.

5. Explica qué es una promesa y escribe un ejemplo.

Una promesa es un objeto que hace un compromiso a ejecutar algo cuando tenga tiempo y te dice si fue exitoso o si fallo.

```
let newPromise = new Promise((resolve, reject) => {  
  let goodResponse = true;  
  if (goodResponse) {  
    resolve("Success!");  
  } else {  
    reject("Failed");  
  }  
});
```

6. Dado el siguiente HTML:

```
<button id="btn">Haz Click</button>
<p id="mensaje"></p>
```

Escribe el JS para que al hacer click en el botón, el <p> muestre "Botón Presionado"

```
const btn = document.getElementById("btn");
const mensaje = document.getElementById("mensaje");
```

```
btn.addEventListener("click", () => {
  mensaje.textContent = "Botón Presionado";
});
```

7. Haz una petición a la API <https://jsonplaceholder.typicode.com/posts> y muestra en consola solo los títulos.

```
fetch("https://jsonplaceholder.typicode.com/posts")
  .then(res => {
    if (!res.ok) {
      throw new Error(`HTTP Error: ${res.status} - ${res.statusText}`);
    }
  })
  .then(posts => {
    const titles = posts.map(post => post.title);
    console.log("Titles: ", titles);
  })
  .catch(err => {
    console.error("Error: ", err.message);
  });
```

8. Explica qué es un closure y escribe un ejemplo

Un closure es una función dentro de otra función que se usa para mantener datos dentro de la nested function privadas al scope de la parent function

```
function createCounter() {
  let count = 0;
  return {
    Increment: () => {
      return count++;
    }
  }
}
```

```
let counter1 = createCounter();  
console.log(counter1.increment()); // 1  
console.log(counter1.increment()); // 2
```

9. ¿Qué es el unit testing y para qué sirve?

Unit testing se usa cuando quieres verificar que una función este ejecutando como queremos y que se pueda automatizar el proceso. Se hacen para ver si aparecen edge cases en forma aislada.

10. Explica qué es una IIFE y escribe un ejemplo.

Una función que se ejecuta al ser definida y es su propia instancia al que sus variables están aisladas del scope afuera de la función y scope global.

```
(function() {  
  const msj = "Hello, world!";  
  console.log(msj);  
})();
```